

402Pilot: An x402 Decision Layer for Autonomous Agent Micropayments

PA-DCT: Payment-Aware Discounted Contextual Thompson Sampling

DSAA 5022

Group 911 Members: Yin LI, Yanbo HE, Wanting XIONG

May 2026

From Payment Execution to Payment Decision

x402 settles. 402Pilot decides which provider to pay.

✓ SOLVED — how to pay

Protocols like x402, AP2, A402 let agents quote, settle, and respond in one HTTP call.

Per-call micropayments are real — and live.

? OPEN — which provider to pay

x402 V2 spec (Coinbase) leaves it to the buyer
"based on historical outcomes."

Fixed budget · unknown quality · drifting prices.

Protocols know how to settle. They don't know who's worth paying.

The Agent-Native Decision Problem

Three constraints define the regime — together they force a fresh design.

01



The agent runs the show

No human sits in the loop. No one resets state, retunes parameters, or kicks a bad provider mid-run.

02



The wallet keeps draining

Every round, the agent watches its remaining budget and adjusts. It's not a hard cap — it's lived experience.

03



Quality and price both shift

Prices can jump overnight. The agent has to learn cost just like it learns quality — both are signals.

Most prior bandits assume a human operator. We don't have one.

Related Work — Four Threads, One Gap

Four threads touch this problem. None covers all three agent-native constraints.



Payment protocols

x402 · AP2 · A2A-x402 · A402

Gives: Settlement, atomicity, identity

Misses: Don't decide who is worth paying



Classical bandits

BTS · DTS · ContextualTS · BwK

Gives: Solid theory for budget + drift

Misses: Not designed for autonomous micropayment buyers



Cost-aware LLM routing

FrugalGPT · RouteLLM · MixLLM

Gives: Quality–cost tradeoff for model choice

Misses: Static price cards · offline · no wallet feedback



Reverse auctions / Agent markets

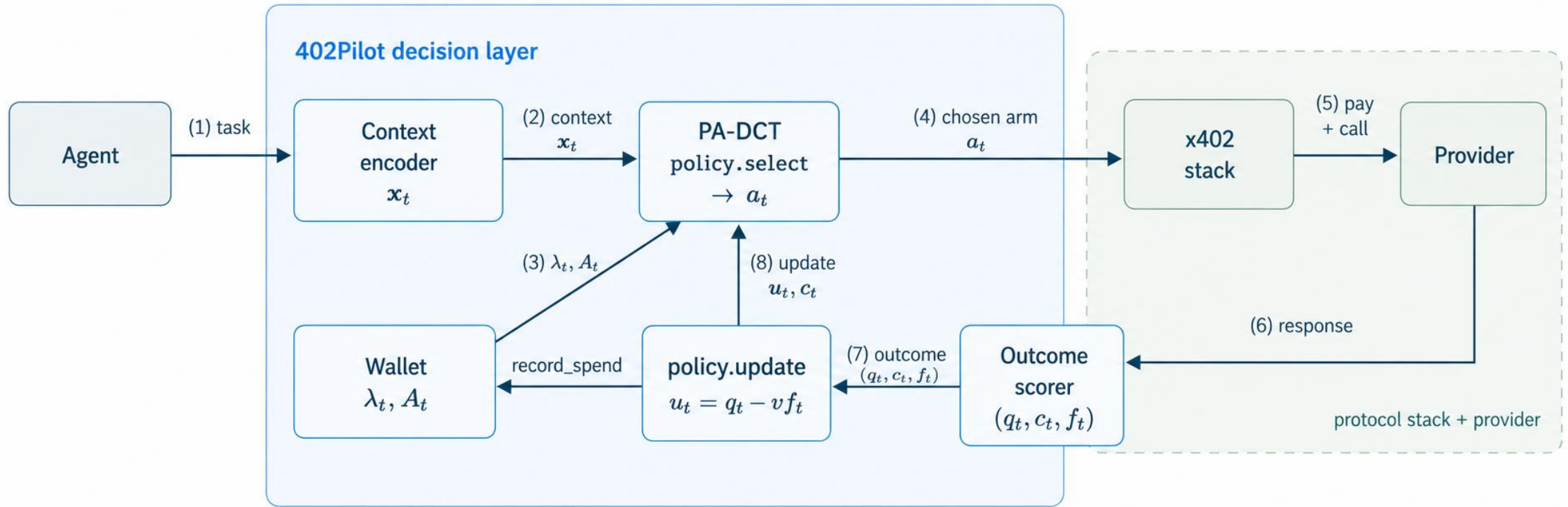
AAMAS '26

Gives: Closest in flavor — provider selection

Misses: Providers BID; we react to provider QUOTES

Bottom line: each thread misses at least one agent-native constraint.

The 402Pilot Decision Layer



Per round:

(1) Task arrives $\rightarrow$ (2) Encode context $x_t \rightarrow$ (3) Check wallet $\lambda_t, A_t \rightarrow$ (4) Choose provider $a_t \rightarrow$ (5) Pay via x402 $\rightarrow$ (6) Receive response $\rightarrow$ (7) Score $q_t, c_t, f_t \rightarrow$ (8) Update with utility u_t and cost c_t , not reward r_t .

x_t = task context; λ_t = budget pressure; A_t = affordable providers; a_t = selected provider; q_t = quality; c_t = cost; f_t = failure flag.

Reward Design: Balancing Quality and Cost

Quality of one call

$$u_t = q_t - v \cdot f_t$$

$v = 0.5$ · failure penalty

Quality minus cost — weighted by wallet pressure

$$r_t = (1 - \bar{\lambda}) u_t - \bar{\lambda} \tilde{c}_t$$

$r_t \in [-1, +1]$

Budget pressure ($\bar{\lambda}$) reacts to spending speed

$$\bar{\lambda} = \lambda_t / (1 + \lambda_t) \in (0, 1)$$

$$\lambda_t = \exp(\alpha \cdot \text{burn_excess}_t), \quad \alpha = 2$$

On track

$$\bar{\lambda} = 0.5$$

balanced quality and cost

Under-spending

$$\bar{\lambda} < 0.5$$

favor quality

Over-spending

$$\bar{\lambda} > 0.5$$

penalize cost more

q_t = quality; f_t = failure flag; u_t = utility; $\tilde{c}_t$ = normalized cost; $\bar{\lambda}$ = wallet pressure weight; r_t = payment-aware reward
 burn_excess_t = how far spending is above plan; α = how fast wallet pressure reacts.

PA-DCT: How the Agent Learns to Choose

Payment-Aware Discounted Contextual Thompson Sampling

Four components; TS balances exploration and exploitation under uncertainty.

P

Payment-aware

$$\pi_a = (1 - \bar{\lambda}) \hat{u}^{a,k} - \bar{\lambda} \hat{c}^{a,k}$$

wallet pressure shifts the tradeoff between quality and cost

D

Discounted

$$(n, S) \leftarrow \gamma (n, S), \quad \gamma = 0.999$$

old observations fade — recent outcomes matter more when the market changes

C

Contextual

$K_b = 4$ task-type buckets per arm

Learn separately for different task types: coding · multi-hop QA · web search open/close

TS

Thompson sampling (TS) — sample, then choose

draw plausible utility + cost from posteriors

explores uncertain providers; avoids greedy lock-in

When price or quality changes, old evidence fades and the agent re-learns.

What PA-DCT Adds Beyond Prior Bandits

We are still a bandit algorithm, but the combination and setting are different.

Prior work comparison

Prior work	Budg.	Disc.?	Ctx.?
BTS [Xia 2015]	✓	✗	✗
DS-TS [Qi 2023]	✗	✓	✗
Contextual TS [Agrawal 2013]	✗	✗	✓
Sliding-Window TS	✗	✓	✗
TV-Reward Contextual TS	✗	✓	✓
C2MAB-V (LLM routing)	partial	✗	✓
NeuralLinUCB	✓	✗	✓
Combinatorial + change-point	✓	✓	✗
PA-DCT (ours)	✓	✓	✓

Our novelty



Setting

agent-native + programmable pricing



Method

budget + discounting + context in one TS policy



System layer

bandits for protocol-agnostic decision layer above x402 micropayments



Benchmark

replay benchmark with controlled market shocks

To our knowledge, prior work does not combine budget, drift adaptation, and context in an autonomous micropayment-buyer setting.

402Pilot-Bench Setup

We freeze real LLM outcomes once, then replay identical task / provider / shock paths — so policy differences come from decisions, not API noise.



823

tasks in our pool

across 4 task buckets



20,575

frozen LLM records

5 providers × 5 versions per task



30 × 10K

seeds × rounds

per (policy, scenario) cell

Four task buckets (drawn from public, fixed sources)

Bucket	Task	Source	Count	Scorer
T1	Coding	HumanEval	164	pass@1
T2	Multi-hop QA	HotpotQA val	220	EM / F1
T3a	Closed web QA	TriviaQA-web	219	EM / F1
T3b	Open web synthesis	OpenAssistant-filtered / custom prompts	220	LLM-as-judge

How we make replay reproducible

1

Prepare

fixed task pool from HumanEval, HotpotQA, TriviaQA-web, and open-ended prompts.

2

Generate

5 sampled responses per (task, provider) via real LLM calls — store response, cost, latency, failure flag.

3

Score & cache

score q once per response, then replay by seed. Bandit loop never re-calls the LLM.

Benchmark Arms & Scenarios

Five providers — three at the same price tier — and three market shocks.

Five provider arms

Provider	Role + model	Behavior	Price / call
P-cheap	<i>low-cost baseline</i> · Qwen3.5-flash	parametric only	\$0.0005
P-mid	<i>reliable workhorse</i> · GPT-5.4-mini	BM25 retrieval	\$0.002
P-premium	<i>high-quality arm</i> · GPT-5.4	CoT + tools	\$0.010
P-adv	<i>deceptive same-tier arm</i> · GPT-5.4-mini	adversarial system prompt	\$0.002
P-flaky	<i>unreliable same-tier arm</i> · GPT-5.4-mini	40% timeout injection	\$0.002

P-mid / P-adv / P-flaky share price and base model — only feedback separates them.

Three calibrated scenarios — what each one tests

S1 Stationary

Learning cost in a stable market.

S2 Mid-tier outage

Reliability adaptation when P-mid fails 30% during rounds 3000–5500.

S3 Premium promo

Price adaptation when P-premium drops to mid-tier price at round 1000.

Baselines and Metrics

5 baselines + an offline upper-bound oracle, evaluated on five locked metrics.

Policy	Exact definition	What it tests
Random	Uniformly samples from affordable providers each round	<i>No-learning lower bound</i>
Always-P-cheap	Always chooses P-cheap	<i>Minimum-spend strategy</i>
Always-P-mid	Always chooses P-mid	<i>Strongest static fixed-arm default</i>
Always-P-premium	Always chooses P-premium until unaffordable	<i>Quality-first, cost-blind strategy</i>
BudgetRule	If wallet >50% → Premium; 20–50% → Mid; ≤20% → Cheap	<i>Hand-written wallet heuristic</i>
Oracle	Hindsight policy that sees realized outcomes before choosing	<i>Upper bound for regret only; not deployable</i>
PA-DCT (ours)	Learns quality and cost per (provider, task bucket) using discounted TS	<i>Adaptive buyer-side policy</i>

Five locked metrics (used in all subsequent slides)

cum-PA	cumulative payment-aware reward — higher is better
task_q	mean response quality — higher is better
ROI	total quality / total spend — high ROI can come from spending very little
Regret	Oracle cum-PA minus policy cum-PA — lower is better
AdaptT	recovery rounds after a shock — shown on the adaptation slide

Main Results: PA-DCT Wins When the Market Changes

PA-DCT pays exploration cost in S1, then becomes the strongest non-oracle policy under shocks.

Policy	S1 cum-PA	S1 task_q	S1 ROI	S1 Regret	S2 cum-PA	S2 task_q	S2 ROI	S2 Regret	S3 cum-PA	S3 task_q	S3 ROI	S3 Regret
Random	3,191	0.688	209	3,646	3,060	0.676	205	3,749	4,382	0.688	370	2,735
AlwaysCheap	5,164	0.610	1,220 [†]	1,673	5,164	0.610	1,220 [†]	1,645	5,164	0.610	1,220 [†]	1,952
AlwaysMid	5,831	0.819	410	1,006	5,069	0.757	379	1,740	5,831	0.819	410	1,286
AlwaysPremium	-3,887	0.866	87	10,725	-3,887	0.866	87	10,696	3,112	0.865	309	4,004
BudgetRule	-82	0.831	208	6,919	-408	0.769	192	7,217	3,064	0.859	307	4,053
PA-DCT	5,512	0.797	378	1,325	5,147	0.761	357	1,662	5,911	0.831	429	1,206
Oracle	6,837	0.901	561	0	6,809	0.901	551	0	7,117	0.906	722	0

Bold green () = best deployable policy in that scenario/metric. Oracle is an offline upper bound, not deployable.*

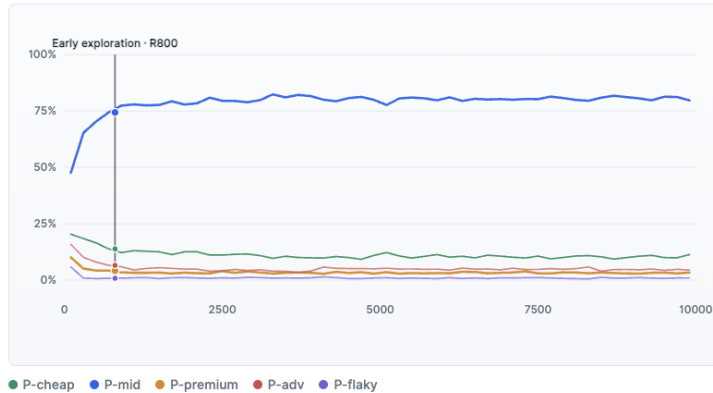
[†] AlwaysCheap's high ROI comes from spending very little — its task_q stays low at 0.610.

AdaptT is omitted (no shock window in S1; shock recovery is shown on the next slide).

Adaptation Under Market Shocks

The reward gains come from changing provider allocation after market changes.

S1 Stable market



No shock: PA-DCT learns a stable allocation, mostly favoring P-mid while still exploring.

S2 Mid-tier outage



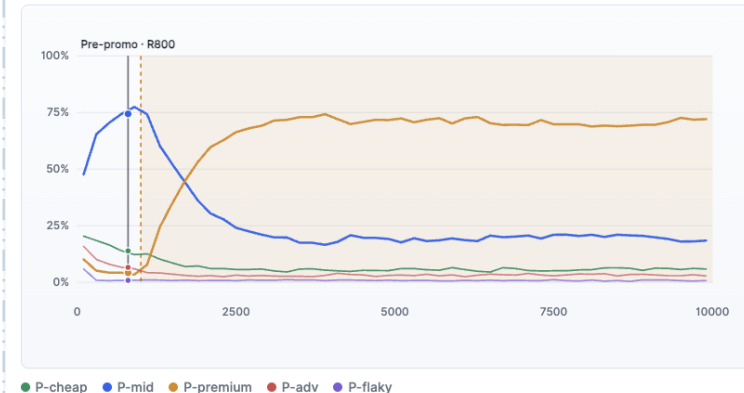
P-mid outage: PA-DCT cuts P-mid usage during rounds 3000–5500, then returns after recovery.

P-mid share **74.9% → 19.4% → 66.3%**

Failure exposure **1.9% vs AlwaysMid 7.5%**

AdaptT **~1,467 rounds**

S3 Premium promo



Premium promo: PA-DCT learns the price drop and shifts spend toward P-premium.

P-premium share **~5% → ~66%**

Price **\$0.01 → \$0.002 at round 1000**

AdaptT **~200 rounds**

Ablation: Each Component Matters

Each component fails on a different metric — so one headline score is not enough.

Removed	What changes	Best evidence
-P Payment-aware	<i>ignores wallet pressure</i>	cum-PA collapses: S1 -1,710; S2 -1,966
-D Discount	<i>old observations do not fade</i>	S2 AdaptT slows: 2,249 vs 1,467
-C Contextual	<i>one posterior across all task types</i>	S3 AdaptT slows: 398 vs 200
-TS Thompson sampling	<i>greedy posterior mean</i>	S3 AdaptT 2,232 vs 200; misses shock in 6/30 seeds
Cost posterior OFF	<i>cost pinned to spec sheet</i>	S3 P-premium share collapses: 60.1% → 3.8%

cum-PA proves payment-awareness · AdaptT proves discount / context / exploration · cost-posterior proves price-shock learning.

Takeaways & Limitations

Three things worth remembering

01

Payment settlement alone does not solve buyer-side provider selection.

02

Good buyer policy must learn quality, failure risk, and cost from feedback.

03

Under market shocks, adaptive spend allocation beats static provider choice.

Limitations

- Tested on a frozen replay dataset and Anvil DEVnet — live deployment still pending.
- Open-ended tasks: P-adv detection limited by LLM-as-judge.
- We study one buyer wallet; multi-agent markets and reputation are future work.

The agent micropayment world is not fully ready yet, but 402Pilot should be ready for the moment it arrives.
<https://github.com/MCCodeAI/402Pilot>